DeepTrustxAI Cloud Security Workshop

DeepTrustxAI

Cloud Security Hands-On Workshop

CCSP foundations, vulnerable cloud deployments and scanner-led remediation

Two-day practitioner training

What You Will Be Able To Do

- Explain cloud security using CCSP and CISSP language.
- Identify how cloud vulnerabilities enter real deployments.
- Deploy a controlled vulnerable lab safely.
- Run CloudGuard scans across cloud, Kubernetes, IaC and web targets.
- Read scanner evidence and convert findings into remediation actions.
- Use open-source tools responsibly without losing traceability.

Workshop Rules

- Use only lab accounts, lab projects and authorized targets.
- Never scan third-party assets without written approval.
- Treat tokens, kubeconfig files and scan outputs as sensitive data.
- Delete temporary vulnerable resources after each lab.
- Keep evidence reproducible: tool, target, timestamp, finding and proof.

Two-Day Learning Path

Day	Focus	Outcome
Day 1 Morning	Cloud security foundations	Understand cloud models, identity, data and network risk
Day 1 Afternoon	Vulnerable deployment lab	Build and inspect an intentionally weak target
Day 2 Morning	CloudGuard scanner usage	Run scans and analyze findings
Day 2 Afternoon	Tools, GRC and remediation	Normalize evidence and close findings

---

1

Module 1

Cloud security and vulnerability concepts

Start with the model before touching the scanner.

Cloud Security In One Sentence

Cloud security is the practice of protecting identities, data, workloads, networks and control-plane permissions across shared infrastructure.

In every cloud incident, ask:

- Who could access it?
- What data or capability was exposed?
- Which control failed?
- Could the attacker move further?
- Can we prove the issue is fixed?

Shared Responsibility Model

Layer	Cloud Provider Handles	Customer Handles
Physical facility	Data center, power, hardware	None

Layer	Cloud Provider Handles	Customer Handles
Cloud platform	Core services, service availability	Secure configuration
Identity	IAM service availability	Users, roles, keys, permissions
Workloads	Runtime primitives	OS, containers, apps, dependencies
Data	Storage durability	Classification, encryption, access

Key point: most findings come from customer-side configuration.

Control Plane vs Data Plane

Plane	What It Controls	Example Risk
Control plane	Creating and configuring resources	Over-permissive IAM role creates public resources
Data plane	Reading, writing and using data	Public bucket exposes sensitive files
Management plane	Logs, billing and admin operations	Disabled audit logs hide attacker activity

If the control plane is weak, the attacker can create more weaknesses.

Identity Is The Perimeter

Cloud identity decides what can be created, read, changed and deleted.

Common identity failures:

- Long-lived access keys.
- No MFA on privileged users.
- Wildcard permissions such as `*:*`.
- Service accounts used by humans.
- Roles trusted by unknown external accounts.
- Secrets stored in code, CI logs or config files.

Network Exposure

Look for paths from the internet to sensitive workloads.

Exposure	Why It Matters
`0.0.0.0/0` on SSH, RDP or database ports	Direct brute-force and exploitation path
Public load balancer to admin panel	Internal tool becomes internet-facing
Missing network policies	Pods or workloads can talk laterally
Wide outbound access	Compromised workload can exfiltrate data

Workshop habit: draw the access path before running tools.

Data Exposure

Data risk usually comes from simple configuration mistakes.

- Public object storage.
- Unencrypted database or disk.
- Secrets inside environment variables.
- Backups without lifecycle or access control.
- Logs containing tokens, PII or credentials.
- Missing retention and deletion policy.

Security is not only "can someone enter"; it is also "what can they take."

Vulnerability Classes

Class	Examples
Identity	Over-permissioned role, stale key, missing MFA
Storage	Public bucket, weak object ACL, no encryption
Network	Public admin port, weak firewall, exposed database
Workload	Vulnerable image, root container, privileged pod
Application	Missing auth, injection, exposed debug route
IaC	Terraform/Helm/Kubernetes manifest creates unsafe default
Operations	No logs, no alerting, no remediation owner

CCSP And CISSP Mapping

Security Concept	CCSP Link	CISSP Link
Shared responsibility	Cloud Concepts and Architecture	Security and Risk Management
IAM and least privilege	Cloud Platform and Infrastructure	Identity and Access Management
Encryption and storage	Cloud Data Security	Asset Security
Secure deployment	Cloud Application Security	Software Development Security
Logging and response	Cloud Security Operations	Security Operations
GRC and evidence	Legal, Risk and Compliance	Security Assessment and Testing

---

Mini Exercise: Read The Risk

For each example, name the vulnerability class and likely impact.

1. Storage bucket allows public read.
2. Kubernetes pod runs as privileged with hostPath mounted.
3. Terraform creates a security group with SSH open to the internet.
4. Service account key is committed to GitHub.
5. Audit logging is disabled for a production account.

Expected output: class, impact, control and one remediation.

2

Module 2

Vulnerable cloud deployment lab

Build a controlled weak target, then learn what each weakness looks like.

Lab Architecture

Component	Lab Weakness	Scanner Should Detect
Object storage	Public read/write or weak ACL	Public storage exposure
IAM user/role	Excessive permissions, stale key	Over-privileged identity
VM or container	Public admin port, vulnerable package	Exposed service, CVE
Kubernetes	Privileged pod, missing network policy	Runtime and RBAC risk
IaC file	Unsafe Terraform or Kubernetes manifest	Misconfiguration before deploy
Web app	Debug endpoint, weak headers	Application security issue

---

Safe Lab Model

Safety Control	How We Use It
Sandbox account/project	No production data or production identity
TTL tag	Resources auto-expire after the lab window
Least privilege deploy role	Lab can create only approved resource types
Budget alert	Stops accidental long-running cost
Destroy step	Cleanup runs after scan completion
Evidence export	Findings remain even after resources are removed

---

Lab Setup Checklist

Before deployment:

- Confirm you are inside the sandbox account/project.
- Confirm CLI profile, region and project ID.
- Confirm no production credentials are loaded.
- Keep terminal logs for the lab.
- Create a cleanup command before creating resources.

During deployment:

- Tag resources with `lab=true` and `ttl_minutes=5`.
- Save generated endpoint, bucket, cluster and IaC paths.

Deploy The Vulnerable App

Instructor demo flow:

1. Show the IaC or Docker Compose file.
2. Point out the intentional insecure settings.
3. Deploy the lab target.
4. Open the application endpoint.
5. Confirm the vulnerable behavior.
6. Start the cleanup timer.

Participant goal: understand the weakness before the scanner reports it.

Find The Weakness Manually

Use this order before scanning:

1. Identity: what role or key is being used?
2. Network: what is reachable from the internet?
3. Data: what storage is exposed?
4. Workload: what runs with high privilege?
5. Code and dependencies: what known issues exist?
6. Logging: can we prove what happened?

Manual review makes scanner output easier to trust.

Hands-On Lab 1

In your assigned lab environment:

1. Deploy the vulnerable target.
2. Record the target name, endpoint and resource IDs.
3. Identify three weaknesses manually.
4. Write the expected scanner finding for each weakness.
5. Keep the cleanup command ready.

Deliverable: a short table of expected findings.

Module 3

CloudGuard scanner usage

Run scans, read evidence and move from finding to action.

CloudGuard Workflow

Stage	What Happens
Target setup	Add credentials, kubeconfig, IaC file or URL
Scan job	Scanner worker runs the selected checks
Evidence	Raw output is normalized into structured records
Findings	Severity, affected asset and proof are stored
GRC mapping	Finding links to control, owner and due date
Closure	Remediation proof is uploaded and validated

Inputs CloudGuard Accepts

Scanner Area	Input
Cloud account	Access key, service account or temporary role
Kubernetes	Kubeconfig with limited read access
IaC	Terraform, Kubernetes YAML, Helm or Compose files
Web app	Authorized URL or internal endpoint
Evidence connector	JSON, CSV, tool output or API payload

Do not scan credentialless unless the target is designed for passive checks.

Run A Cloud Scan

1. Open CloudGuard.
2. Go to credentials or target setup.
3. Select the scanner type.
4. Choose sandbox account or project.
5. Start the scan.
6. Watch job status: queued, running, completed or failed.
7. Open findings and filter by severity.

8. Export evidence for the lab report.

Run A Kubernetes Scan

1. Upload or select kubeconfig.
2. Confirm namespace scope.
3. Run RBAC, pod, service and network policy checks.
4. Review privileged pods, public services and risky roles.
5. Compare live cluster findings with Kubernetes manifest files.
6. Mark findings that need runtime proof.

Important: kubeconfig should be read-only for workshop scans.

Run An IaC Scan

1. Upload the full Terraform, Kubernetes YAML, Helm or Compose file.
2. Select scanner rules.
3. Scan before deployment.
4. Review unsafe defaults.
5. Fix the IaC file.
6. Scan again.
7. Compare before and after evidence.

IaC scanning catches mistakes before they become cloud resources.

Run A Web App Scan

1. Enter the authorized lab URL.
2. Start passive checks first.
3. Run active checks only against the lab target.
4. Review missing headers, exposed paths and vulnerable endpoints.
5. Save proof without leaking secrets.
6. Link the finding to the affected app or API.

Never run active tests against public third-party targets.

Read A Finding

Every useful finding needs:

Field	Meaning
Asset	What is affected
Severity	How urgent it is
Proof	Why the scanner believes it
Impact	What could happen
Control	Which policy or standard it maps to
Owner	Who must fix it
Remediation	What action closes it
Validation	How we prove it is fixed

Evidence Format

```
{
  "asset": "lab-storage-bucket",
  "scanner": "cloudguard-storage",
  "finding": "Public object storage access",
  "severity": "high",
  "proof": "Bucket policy allows anonymous read",
  "control": "Cloud Data Security",
  "remediation": "Remove public ACL and enforce private policy",
  "validation": "Re-scan shows anonymous access denied"
}
```

GRC Closure Flow

Step	Action
Create	Finding is converted into a GRC record
Assign	Owner and due date are added
Fix	Team applies remediation
Validate	Scanner or manual proof confirms closure
Archive	Evidence remains for audit

The goal is not only finding issues. The goal is proving closure.

Hands-On Lab 2

Use the lab target from Module 2.

1. Run one cloud scan.
2. Run one IaC scan.
3. Run one web scan.
4. Select the top three findings.
5. Convert each finding into the evidence format.
6. Add owner, remediation and validation method.

Deliverable: three complete evidence records.

4

Module 4

Open-source tools and scanner coverage

Understand what each tool is good at and where it can mislead you.

Tool Coverage Map

Category	Tools	Best For
Cloud posture	Prowler, ScoutSuite, CloudFox	Account-level misconfiguration
IaC scanning	Checkov, KICS, Terrascan, tfsec	Pre-deployment mistakes
Container	Trivy, Grype	Image CVEs and package risk
Secrets	Gitleaks, TruffleHog	Exposed credentials
Web testing	OWASP ZAP, Nuclei	Web app and template-based checks
Kubernetes	Kubescape, kube-bench, Polaris	Cluster and manifest hardening
Code review	Semgrep, Bandit	Source-level security issues

Tool Strengths And Limits

Tool Type	Strength	Limit
Cloud posture	Broad account coverage	Needs correct credentials and context
IaC scanner	Fast prevention	May miss runtime drift
Container scanner	Strong CVE visibility	CVE does not always mean exploitable
Secret scanner	Finds leaked keys	Needs careful redaction
Web scanner	Finds exposed web issues	Active scans can be disruptive
Kubernetes scanner	Good for RBAC and pod risks	Needs cluster access

---

Normalize Tool Output

Raw tool output becomes useful only after normalization.

For every tool result, extract:

- Asset name and type.
- Rule or check ID.
- Severity.
- Human-readable proof.
- Recommended fix.
- Validation command or re-scan condition.
- Control mapping.

This is how separate tools feed one GRC process.

False Positive Handling

Do not delete a finding just because it looks inconvenient.

Use one of these decisions:

Decision	Meaning
Valid	Must be fixed
Accepted risk	Business owner accepts the exposure
False positive	Evidence proves no exposure
Duplicate	Same issue already tracked
Needs validation	More proof required

Every decision needs a reason.

Hands-On Lab 3

You will receive sample outputs from different tools.

1. Identify the affected asset.
2. Extract the proof.
3. Normalize the finding.
4. Map it to CCSP or CISSP language.
5. Decide remediation and validation.

Deliverable: one normalized evidence record per tool.

Capstone

Each team demonstrates one complete security story.

Step	What To Show
Weakness	Where the vulnerability was introduced
Scan	Which scanner or tool found it
Evidence	What proof was captured
Risk	Why it matters
Fix	What changed
Validation	How closure was proven

Final Checklist

Before leaving the workshop:

- Destroy lab resources.
- Confirm no public lab endpoints remain.
- Remove temporary credentials.
- Export scan evidence.
- Record the top lessons learned.
- Keep the normalized evidence template for future projects.

Security work is complete only when the vulnerable path is closed and evidence proves it.